

Lab 3: MATLAB Programming

Authors	Name: 何衍宁 Name: 夏天	Student ID:12311512 Student ID:12311610
Objective Lab3 In Lab 3, we primarily focus on building our proficiency with MATLAB to analyze and synthesize periodic signals using discrete-time Fourier series (DTFS) and fast Fourier transform (FFT). By utilizing the fft and freqz functions, we gain insights into the computational processes for calculating frequency responses of linear time-invariant (LTI) systems. Additionally, the tutorial guides us through simulating system responses using lsim, exploring the eigenfunction properties of complex exponentials, and evaluating the effects of simple filters on signals. These skills not only enhance our understanding of signal processing concepts but also equip us with practical experience in employing MATLAB for real-world applications.. Lab results & Analysis 3.8 This exercise demonstrates the effect of first-order recursive discrete-time filters on periodic signals. You will examine the frequency responses of two different systems and also construct a periodic signal to use as input for these systems. This exercise assumes you are familiar with using fft and ifft to compute the DTFS of a periodic signal as described in Tutorial 3.1. In addition, it is also assumed you are proficient with the filter and freqz commands described in Tutorials 2.2 and 3.2. Several parts of this exercise require you to generate vectors which should be purely real, but have very small imaginary parts due to roundoff errors. Use real to remove these residual imaginary parts from these vectors. This exercise focuses on two causal LTI systems described by first-order recursive difference equations: System 1: $y[n] - 0.8y[n-1] = x[n],$ System 2: $y[n] + 0.8y[n-1] = x[n].$ The input signal $x[n]$ will be the periodic signal with period $N = 20$ described by the DTFS coefficients $a_k = \begin{cases} 3/4, & k = \pm 1, \\ -1/2, & k = \pm 9, \\ 0, & \text{otherwise.} \end{cases} \quad (3.10)$		

3.8 a

Intermediate Problems

(a). Define vectors **a1** and **b1** for the difference equation describing System 1 in the format specified by **filter** and **freqz**. Similarly, define **a2** and **b2** to describe System 2.

Define **a1**, **b1** to be the coefficients of the filter and **freqz** functions of system 1, **a2**, **b2** to be the coefficients of the filter and **freqz** functions of system 2, representing **a1**, **a2**, **b1**, **b2**

Analysis:

Let's write it in terms of the definition of **b** and **a**.

```
clc;

clear;

a1 = [1,-0.8]

b1 = [1]

a2 = [1,0.8]

b2 = [1]
```

```
a1 = 1×2
    1.0000   -0.8000

b1 = 1

a2 = 1×2
    1.0000    0.8000

b2 = 1
```

Lab result:

a1=[1 -0.8]; **b1**=[1 0]; **a2**=[1 0.8]; **b2**=[1 0];

3.8 b

(b). Use **freqz** to evaluate the frequency responses of Systems 1 and 2 at 1024 points between 0 and 2π . Note that you will have to use the 'whole' option with **freqz** to do this. Use plot and abs to generate appropriately labeled graphs of the magnitude of the frequency response for both systems. Based on the frequency response plots, specify whether each system is a highpass, lowpass, or bandpass filter.

The freqz function is used to find the frequency response of system 1 and system 2

Analysis:

With the freqz method, the frequency is high near odd π and low near even π

```
clear;

clc;

a1 = [1,-0.8]

b1 = [1]

a2 = [1,0.8]

b2 = [1]

figure

a = linspace(2,2*pi,1024)

[H1,a] = freqz(b1,a1,1024,'whole')

[H2,a] = freqz(b2,a2,1024,'whole')

subplot(2,1,1),plot(a,abs(H1))

legend('H1(W)')

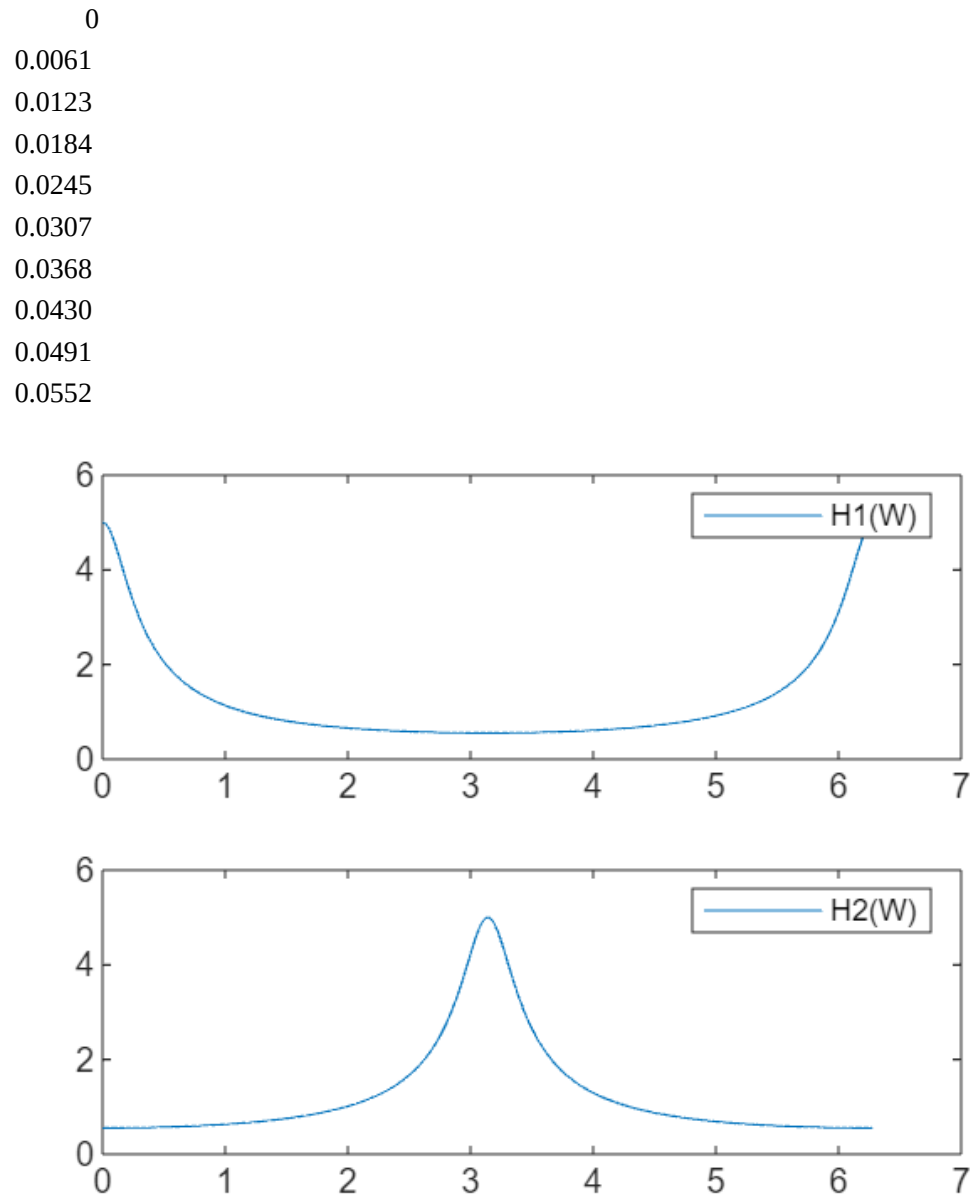
subplot(2,1,2),plot(a,abs(H2))

legend('H2(W)')
```

```
a1 = 1×2
    1.0000    -0.8000
b1 = 1
a2 = 1×2
```

Signals and Systems

```
1.0000    0.8000
b2 = 1
a = 1×1024
    2.0000    2.0042    2.0084    2.0126    2.0167    2.0209    2.0251    2.0293
2.0335    2.0377    2.0419    2.0461    2.0502    2.0544    2.0586    2.0628
2.0670    2.0712    2.0754    2.0796    2.0837    2.0879    2.0921    2.0963
2.1005    2.1047    2.1089    2.1130    2.1172    2.1214    2.1256    2.1298
2.1340    2.1382    2.1424    2.1465    2.1507    2.1549    2.1591    2.1633
2.1675    2.1717    2.1758    2.1800    2.1842    2.1884    2.1926    2.1968
2.2010    2.2052
H1 = 1024×1 complex
    5.0000 + 0.0000i
    4.9966 - 0.1226i
    4.9865 - 0.2447i
    4.9697 - 0.3657i
    4.9464 - 0.4850i
    4.9169 - 0.6022i
    4.8812 - 0.7167i
    4.8399 - 0.8282i
    4.7931 - 0.9362i
    4.7414 - 1.0405i
a = 1024×1
    0
    0.0061
    0.0123
    0.0184
    0.0245
    0.0307
    0.0368
    0.0430
    0.0491
    0.0552
H2 = 1024×1 complex
    0.5556 + 0.0000i
    0.5556 + 0.0015i
    0.5556 + 0.0030i
    0.5556 + 0.0045i
    0.5556 + 0.0061i
    0.5556 + 0.0076i
    0.5556 + 0.0091i
    0.5556 + 0.0106i
    0.5556 + 0.0121i
    0.5556 + 0.0136i
a = 1024×1
```

**Lab result:**

As can be seen from the figure, system 1 is low-pass filtering, and system 2 is high-pass filtering

3.8 c

(c). Use Eq. (3.10) to define the vector a_k to be the DTFS coefficients of $x[n]$ for $0 \leq k \leq 19$. Generate a plot of the DTFS coefficients using stem where the x-axis is labeled with frequency $\omega_k = (2\pi/20)k$. Compare this plot with the frequency responses you generated in Part (b), and for each system state which frequency components will be amplified and which will be attenuated when $x[n]$ is the input to the system.

Analysis:

Define a_x as a Fourier series for $x[n]$, draw a picture of a_x at $\omega_k = (2\pi/20)k$, and determine which part of the frequency is compared to the picture of problem b is enlarged, which part is reduced.

```
clear;

clc;

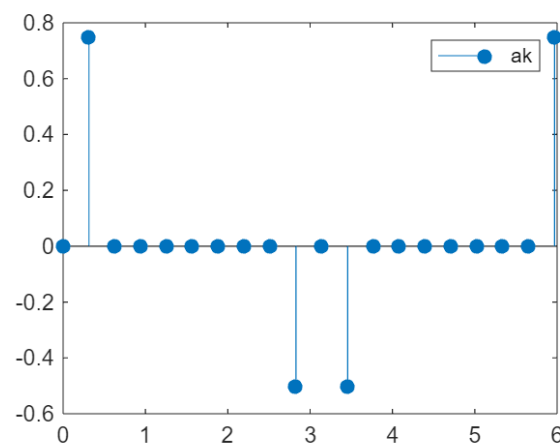
figure

ak = [0 0.75 zeros(1,7) -0.5 0 -0.5 zeros(1,7) 0.75];

wk = 0:pi/10:(pi/10*19);

stem(wk,ak,'filled')

legend('ak')
```



Lab result:

For high-pass systems, the frequency is amplified when k is odd and decreases when it is even. For low-pass systems, the frequency is amplified when k is even and decreases when it is odd. When $x[n]$ is used as the input signal, ω_1 and ω_{19} will be amplified, ω_9 and ω_{11} will be weakened, and ω_1 and ω_{19} will be weakened, and ω_9 and ω_{11} will be amplified for system 2.

3.8 d

(d). Define x_{-20} to be one period of $x[n]$ for $0 \leq n \leq 19$ using **ifft** with a_x as specified in Tutorial 3.1. Use x_{-20} to create x , consisting of 6 periods of $x[n]$ for $-20 \leq n \leq 99$. Also define n to be this range of sample indices, and generate a plot of $x[n]$ on this interval using **stem**.

Define x_{-20} as the set of vectors $x[n]$ in the range 0 to 19, find the set x of vectors $x[n]$ in the range -20 to 99, and plot $x[n]$.

Analysis:

I use the **ifft** function to find x_{-20} .

```
clear;

clc;

figure

ak = [0 0.75 zeros(1,7) -0.5 0 -0.5 zeros(1,7) 0.75];

wk = 0:pi/10:(pi/10*19);

x_20 = (20).*ifft(ak)

xn = [x_20 x_20 x_20 x_20 x_20 x_20];

subplot(2,1,1);

stem(0:19,x_20,'k','filled');

legend('x20[n]')

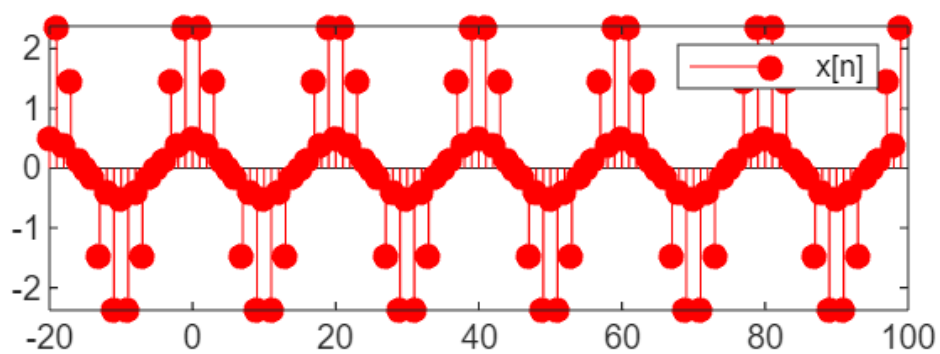
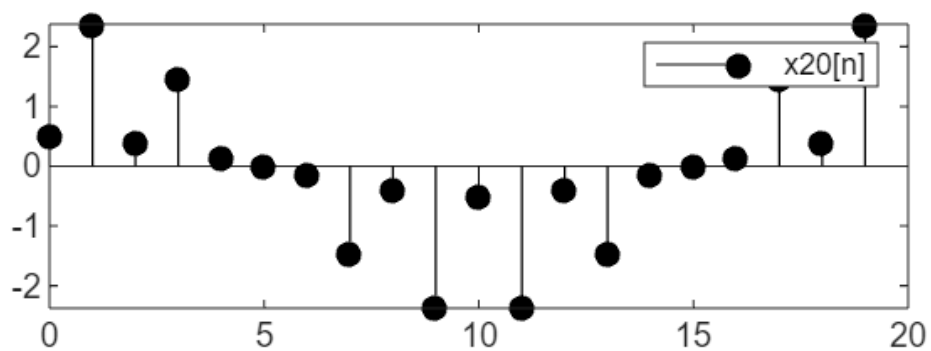
subplot(2,1,2);

stem(-20:99,xn,'r','filled');
```

legend('x[n]')

$x_{20} = 1 \times 20$

0.5000	2.3776	0.4045	1.4695	0.1545	-0.0000	-0.1545
-1.4695	-0.4045	-2.3776	-0.5000	-2.3776	-0.4045	-1.4695
-0.1545	0.0000	0.1545	1.4695	0.4045	2.3776	



Lab result:

Figure is drawn.

3.8 e

(e). Use filter to compute y_1 and y_2 , the outputs of Systems 1 and 2 when $x[n]$ is the input. Plot both outputs for $0 \leq n \leq 99$ using stem. Based on these plots, state which output contains more high frequency energy and which has more low frequency energy. Do the plots confirm your answers in Part (c)?

Calculate the output y_1 and y_2 of system 1,2, using the filter function, and draw a picture stating which contains more high-frequency energy

Analysis:

Draw the images. Whichever is denser is more frequent.

```
ak = [0 0.75 zeros(1,7) -0.5 0 -0.5 zeros(1,7) 0.75];
```

```
wk = 0:pi/10:(pi/10*19);
```

```
a1 = [1,-0.8];
```

```
b1 = [1];
```

```
a2 = [1,0.8];
```

```
b2 = [1];
```

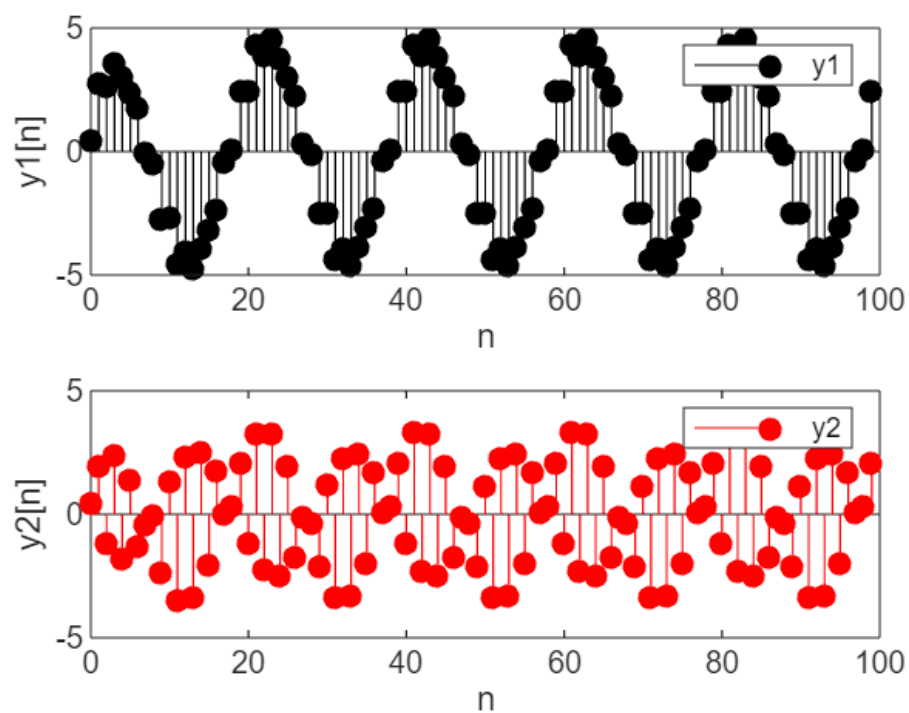
```
x_20 = (20).*fft(ak);
```

```
xn = [x_20 x_20 x_20 x_20 x_20 x_20];
```

```
n=0:99;
```

```
x=[x_20 x_20 x_20 x_20 x_20];
```

```
y1=filter(b1,a1,x);  
  
y2=filter(b2,a2,x);  
  
figure  
  
subplot(2,1,1);  
  
stem(0:99,y1,'k','filled');  
  
xlabel('n');  
  
ylabel('y1[n]');  
  
legend('y1')  
  
subplot(2,1,2);  
  
stem(0:99,y2,'r','filled');  
  
xlabel('n');  
  
ylabel('y2[n]');  
  
legend('y2')
```



Lab result:

y2 has a higher frequency and more high frequency energy

3.8 f

(f). Define y1-20 and y220 to be the segments of y1 and y2 corresponding to y1[n] and y2[n] for $0 < n < 19$. Use these vectors and **fft** to compute a-y1 and a-y2, the DTFS coefficients of y1 and y2. Use stem and abs to generate plots of the magnitudes of the DTFS coefficients for both sequences. Do these plots agree with your answers in Part (e)?

The fft function is used to compute the Fourier series of y1 and y2 in the range 0-19 and draw a graph

Analysis:

Use fft.

```
clc;

clear;

ak = [0 0.75 zeros(1,7) -0.5 0 -0.5 zeros(1,7) 0.75];

wk = 0:pi/10:(pi/10*19);

a1 = [1,-0.8];

b1 = [1];

a2 = [1,0.8];

b2 = [1];

x_20 = (20).*ifft(ak);

xn = [x_20 x_20 x_20 x_20 x_20 x_20];

n=0:99;

x=[x_20 x_20 x_20 x_20 x_20];

y1=filter(b1,a1,x);
```

```
y2=filter(b2,a2,x);

ky=0:pi/10:(pi/10*19);

y1_20=y1(1:20);

y2_20=y2(1:20);

a_y1=1/20.*fft(y1_20);

a_y2=1/20.*fft(y2_20);

figure;

subplot(2,1,1);

stem(ky,abs(a_y1),'k','filled');

xlabel('ωk');

ylabel('a_y1');

legend('a_y1');

title('figure1');

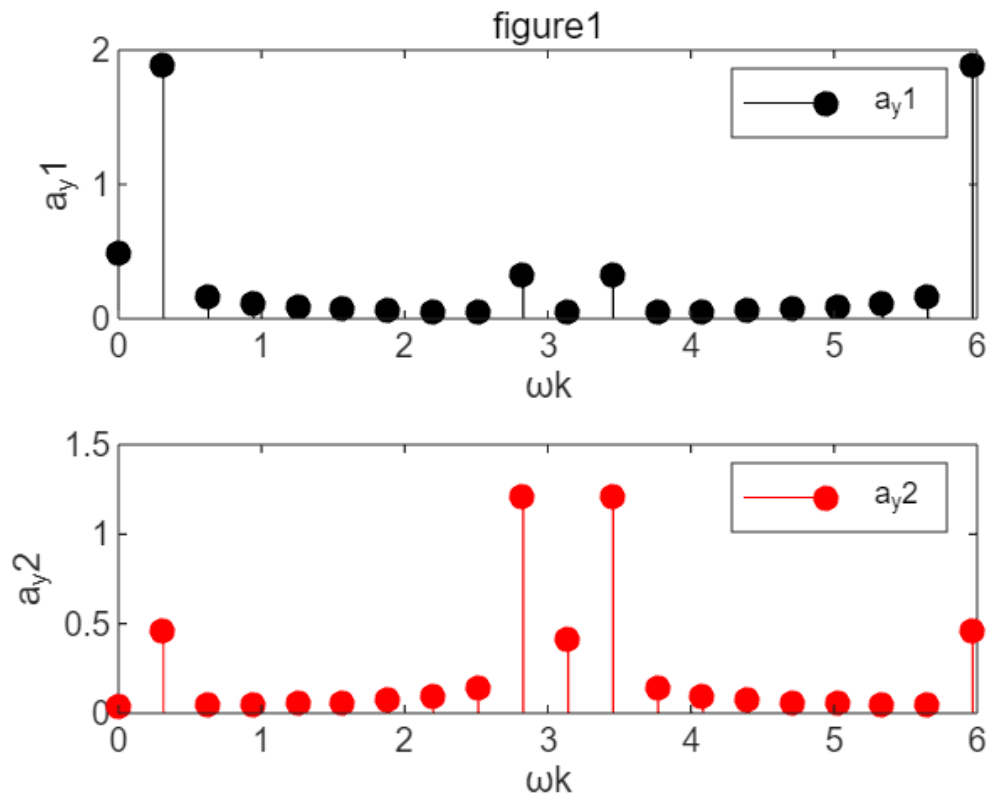
subplot(2,1,2);

stem(ky,abs(a_y2),'r','filled');

xlabel('ωk');

ylabel('a_y2');

legend('a_y2');
```



Lab result:

The discrete time Fourier series coefficients of y_1 and y_2 are drawn as shown above. According to the drawn images, the conclusion is consistent with that in question (e), y_1 contains more low frequency energy and y_2 contains more high frequency energy.

3.10 a

Advanced Problems

(a). Argue that computing each coefficient a_k requires $N + 1$ complex multiplications and $N - 1$ complex additions. Assume that the functions $e^{-jk(2\pi/N)n}$ do not require any computation. Using this result, how many operations are required to compute the DTFS for a signal with fundamental period N ? Note that the number of operations required is independent of the particular signal $x[n]$.

While the DTFS is useful for analyzing many signals and systems, its popularity is due in part to the existence of a fast algorithm known as the Fast Fourier Transform (FFT). Before the landmark paper by Cooley and Tukey was published in 1965, the fastest algorithms used at the time for computing the DTFS coefficients required $O(N^2)$ operations. ($O(N^2)$ operations means that the number of complex additions and multiplications required is a polynomial in N of order 2. When characterizing the growth in computational complexity of an algorithm, the lower order terms can be ignored, since they become negligible for large N .) The FFT algorithm proposed by Cooley and Tukey, however, requires only O_1 .

($N \log N$) operations. For large N , the computational savings can be tremendous; e.g., for $N = 800$, compare $N^2 = 1 \times 10^6$ with $N \log N = 3 \times 10^3$. The difference is three orders of magnitude. The FFT essentially allows Fourier analysis to be applied to signals with very large fundamental periods. In the following problem, you will plot the number of operations required by the FFT and compare them with an $O(N^2)$ algorithm.

For the next two parts, you should assume $x[n]$ has fundamental period N and takes on the values $x[n] = (0.9)^n$ on the interval $0 \leq n \leq N - 1$.

They want to figure out the total number of calculations in the process of finding the DTFS

Analysis:

According to the Fourier series calculation formula, to calculate each a_k , we need to multiply $x[n]$ by e^{-jwkn} a total of N times, and divide by N , a total of $N+1$ multiplications.

All N calculations are then added together for a total of $n-1$ operations, requiring a total of $2N$ calculations. There are N a_k s to calculate. So the total is $2 \cdot N \cdot N = 2N^2$

This question needs no code.

Lab result:

The total value of DTFS is $2N^2$.

3.10 b

(b). If you have not already done the Advanced Problem in Exercise 3.5 writing `dtfs`, do so now. You will compare the amount of computation this algorithm requires with the

¹J.W. Cooley and J.W. Tukey, "An algorithm for the machine calculation of complex Fourier series", *Mathematics of Computation*, 1965, 19(89-92), pp. 297-301.

amount required by `fft`. You can measure the number of operations used by `dtfs` to implement Eq. (3.2) by using the internal `flops` (floating point operations, i.e., additions and multiplications) counter of MATLAB as follows:

```
>> x = (0.9).^[0:N-1]; % create one period of x[n]
>> flops(0); % set MATLAB's internal computation counter to 0
>> X = dtfs(x,0); % Store the DTFS of x[n] in X
>> c = flops; % Store the number of operations in c
```

Find `c` for computing `X` using `dtfs` for $N = 8, 32, 64, 128$, and 256 . Save these values in the vector `dtfscomps`.

The problem requires to calculate the computation amount of FS calculated by the custom algorithm DTFS when $x[n] = (0.9)^n$, $N=8,32,64,128,256$, and write the result into the `dtfscomps` array.

Analysis:

The `flops` function algorithm is inefficient and negligent, here the `timeit` method is used, $x[n]$ is written and calculated. To improve accuracy, each N value is calculated 50 times and averaged in milliseconds. Then, $\log_2(N)$ is the horizontal coordinate and t is the vertical coordinate.

```
clc;

clear;

figure
```

```
N0=[8,32,64,128,256];

dtfscomps=zeros(1,5);

function [X, x_reconstructed] = dtfs(x, synthesis_flag)

    N = length(x);

    X = zeros(1, N);

    for k = 0:N-1

        for n = 0:N-1

            X(k+1) = X(k+1) + x(n+1) * exp(-1j * 2 * pi * k * n / N);

        end

    end

    if synthesis_flag

        x_reconstructed = zeros(1, N);

        for n = 0:N-1

            for k = 0:N-1

                x_reconstructed(n+1) = x_reconstructed(n+1) + (X(k+1) / N) * exp(1j * 2 * pi * k * n / N);

            end

        end

    else

        x_reconstructed = [];

    end

end

for i = 1:length(N0)
```



```
N=N0(i);

x = 0.9.^(0:N-1);

t=0;

for j=0:49

    f = @()dtfs(x,false);

    t_timeit = timeit(f);

    t = t+t_timeit;

end

t = t/50;

dtfscomps(i)=t*1000;

end

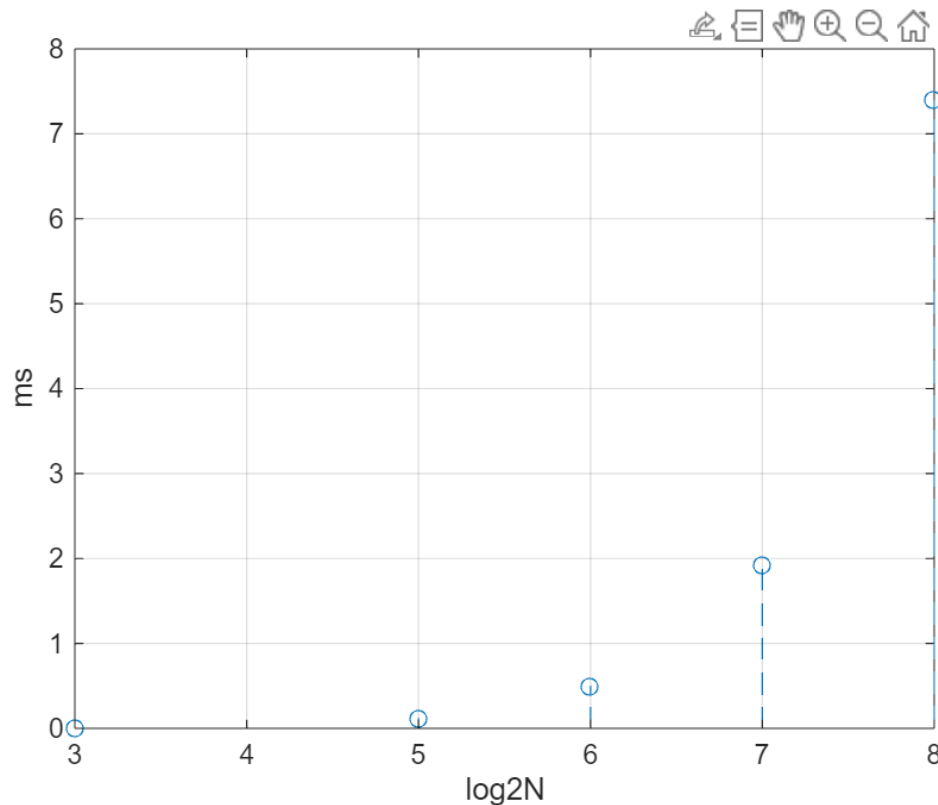
N1 = [3,5,6,7,8];

stem(N1,dtfscomps,'LineStyle','--')

xlabel('log2N')

ylabel('ms')

grid on
```



Lab result:

It can be seen from the figure that the calculation time becomes longer after N increases, and it basically satisfies the formula $2 \cdot N^2$, which is consistent with the analysis in the above question.

3.10 c

(c). Now, compute the DTFS coefficients of $x[n]$ for $N = 8, 32, 64, 128$, and 256 using `fft` as shown in Tutorial 3.1. Use `flops` to find the number of operations used for each value of N and store these values in the vector `fftcomps`. Plot `dtfscomps` and `fftcomps` versus N using `loglog`. How does the number of operations required by `fft` compare to that required by `dtfs`, particularly for large values of N ?

The problem requires to find the fft calculation of Fourier series when $N = 8, 32, 64, 128, 256$, save the results in the `fftcomps` array; Then the `loglog` method is used to plot `dtfscomps` and `fftcomps` against N on the same graph.

Analysis:

As with b, using the `timeit` method, write `x[n]` and calculate. To improve accuracy, each N value is calculated 50 times and averaged in milliseconds.

figure

```
N0=[8,32,64,128,256];
```

```
fftcomps=zeros(1,5);
```

```
function [X, x_reconstructed] = dtfs(x, synthesis_flag)
```

```
N = length(x);
```

```
X = zeros(1, N);
```

```
for k = 0:N-1
```

```
for n = 0:N-1
```

```
X(k+1) = X(k+1) + x(n+1) * exp(-1j * 2 * pi * k * n / N);
```

```
end
```

```
end
```

```
if synthesis_flag
```

```
x_reconstructed = zeros(1, N);
```

```
for n = 0:N-1
```

```
for k = 0:N-1
```

```
x_reconstructed(n+1) = x_reconstructed(n+1) + (X(k+1) / N) * exp(1j * 2 * pi * k * n / N);
```

```
end
```

```
end
```

```
else
```

```
x_reconstructed = [];
```

```
end
```

```
end
```

```
dtfscomps=zeros(1,5);

for i = 1:length(N0)

    N=N0(i);

    x = 0.9.^(0:N-1);

    t=0;

    for j=0:49

        f = @()dtfs(x,0);

        t_timeit = timeit(f);

        t = t+t_timeit;

    end

    t = t/50;

    dtfscomps(i)=t*1000;

end


for i = 1:length(N0)

    N=N0(i);

    x = 0.9.^(0:N-1);

    t=0;

    for j=0:49

        f = @()fft(x,0);

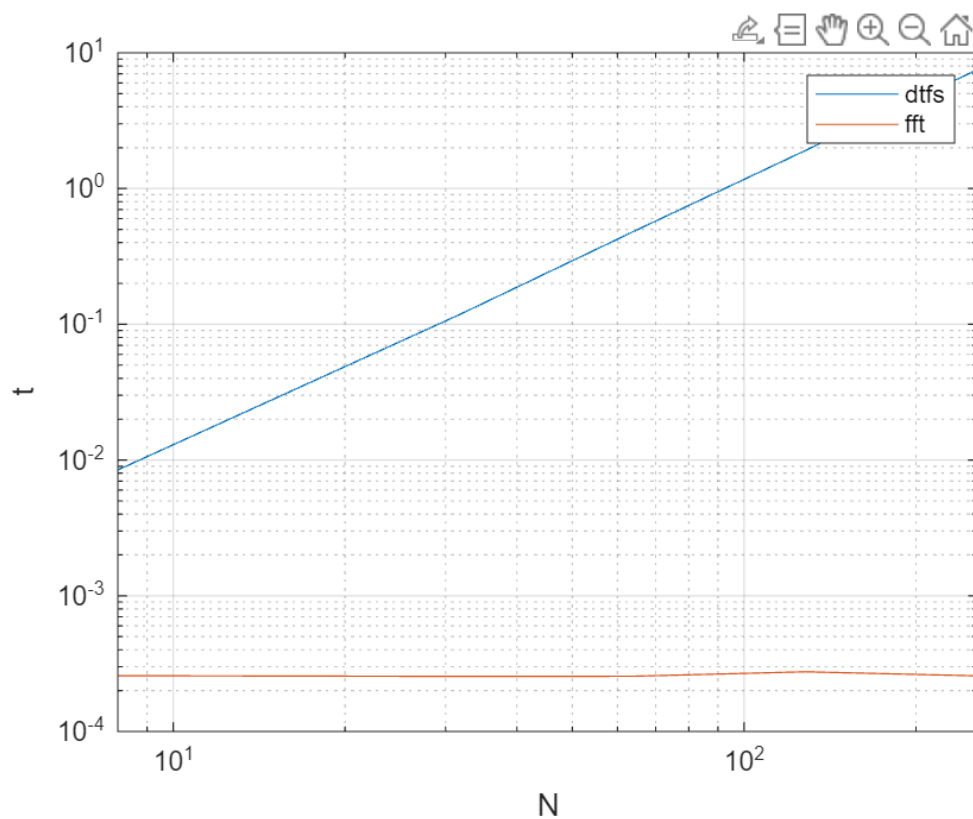
        t_timeit = timeit(f);

        t = t+t_timeit;

    end

    t = t/50;
```

```
fftcomps(i)=t;  
  
end  
  
fftcomps = fftcomps.*(1000);  
  
loglog(N0,dtfscomps,N0,fftcomps)  
  
grid on  
  
xlabel('N')  
  
ylabel('t')  
  
legend('dtfs','fft');
```



Lab result:

Figure is drawn. As can be seen from the figure, after using the fast Fourier transform, the computation amount N is greatly reduced, and with the increase of N , the computation amount using the custom method increases greatly, while the computation amount using the fft method hardly increases.

3.10 d

If computational savings is defined as the ratio of the operations required by the slow algorithm to those of the fast algorithm, note that the savings provided by the FFT increase as N increases (at the rate of roughly $N/\log N$).

Periodic Convolution with the FFT

In many applications, LTI systems are implemented using periodic convolution. In the following problems, you will implement the periodic convolution of two discrete-time signals using two different methods, one of which takes advantage of the computational savings provided by the FFT. The periodic convolution of two periodic discrete-time signals $x[n]$ and $h[n]$, both with fundamental period N , is

$$y[n] = \sum_{r=0}^{N-1} x[r]h[n-r]. \quad (3.11)$$

- (d). What is the fundamental period, N_y , of $y[n]$? Argue that directly implementing the periodic convolution according to Eq. (3.11) requires $\mathcal{O}(N^2)$ operations (additions and multiplications). Remember that computing one period of $y[n]$ is sufficient to characterize the entire signal.

Both $h[n]$ and $x[n]$ functions are known to be signals of period N . The period and computation amount of the periodic convolution function are required.

Analysis:

Since $y[n+N] = x[n] * h[n+N] = x[n] * h[n] = y[n]$ therefore $y[n]$ is a signal of period N , computing an item in the convolution requires $n-1$ addition and n -multiplication, so the amount of computation after computing one period is $N * (n-1) + N * N = 2N^2 - N$. It corresponds to the computation of $\mathcal{O}(N^2)$.

This question needs no code.

Lab result:

$y[n]$ is a signal with period N , and the total number of operations is $2N^2 - N$

3.10 e

(e). Assume both $x[n]$ and $h[n]$ have fundamental period $N = 40$, and are given by $x[n] = (0.9)^n$ and $h[n] = (0.5)^n$ over the interval $0 \leq n \leq N - 1$. Compute the periodic convolution of $x[n]$ with $h[n]$ and plot $y[n]$ for $0 \leq n \leq N_y - 1$. Store the number of operations, given by `flops`, required to implement the convolution in `f40c`. Remember to set `flops(0)` after creating `x` and `h`, the vectors representing $x[n]$ and $h[n]$. Hint: To implement the periodic convolution, first store $x[n]$ and $h[n]$ over the interval $0 \leq n \leq N - 1$ in the row vectors `x` and `h`, respectively. Set `flops(0)` and then use `conv([x x],h)`. The periodic convolution can be extracted from a portion of this signal.

The problem asks to calculate the result of the periodic convolution of two functions $x[n]=0.9^n$ and $h[n]=0.5^n$ when $N=40$, and record the running time.

Analysis:

Just write the code as required. Since use `timeit` instead of `flops`, we use `time` to replace the number of operations.

```
clc;

clear;

figure

xn4 = 0:39;

xn = 0.9.^xn4;

hn = 0.5.^xn4;

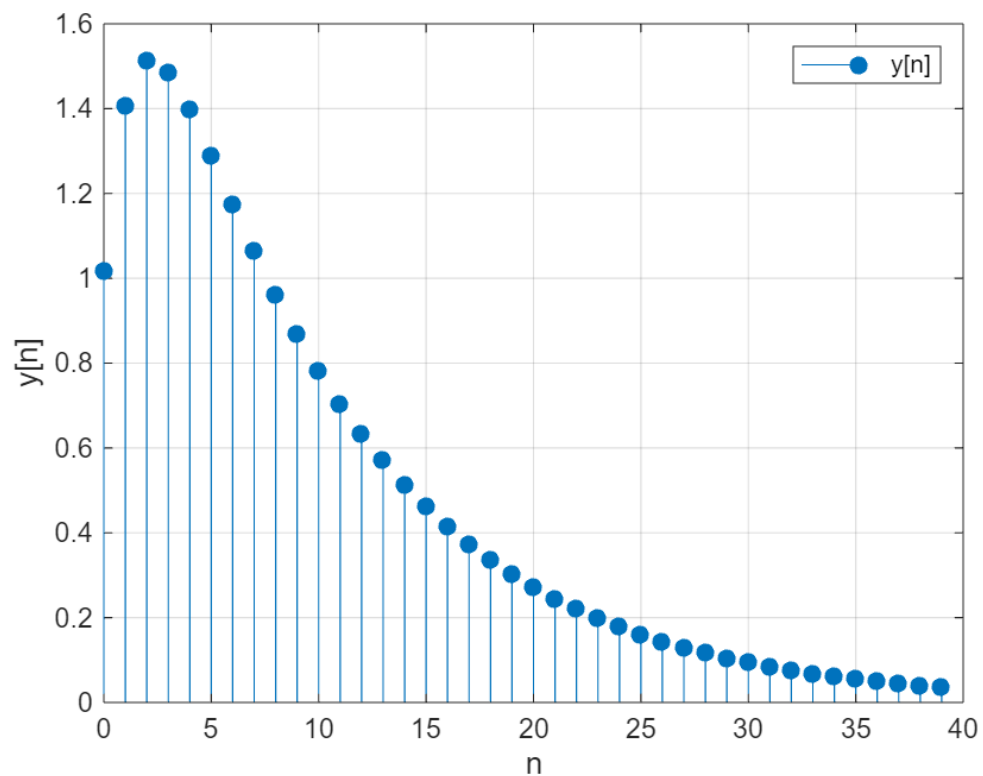
zn = conv([xn xn],hn);

time = 0;

for i = 0:1:49

    f = @(conv([xn xn],hn));
```

```
t_timeie2 = timeit(f);  
  
time = time+t_timeie2;  
  
end  
  
yn=zn(41:80);  
  
stem(xn4,yn,'filled');  
  
grid on  
  
xlabel('n');  
  
ylabel('y[n]')  
  
legend('y[n]')  
  
fprintf('执行时间（秒）： %.6f\n', time/50);  
  
f40c = time/50
```



执行时间（秒）： 0.000001

f40c = 1.4519e-06

lab result:

figure is drawn.

3.10 f

(f). Repeat Part (e) for $N = 80$, again plotting a period of $y[n]$ and storing the number of operations in **f80c**.

They want the same thing as e, but instead of the period being $N=80$

Analysis:

Same as e.

figure

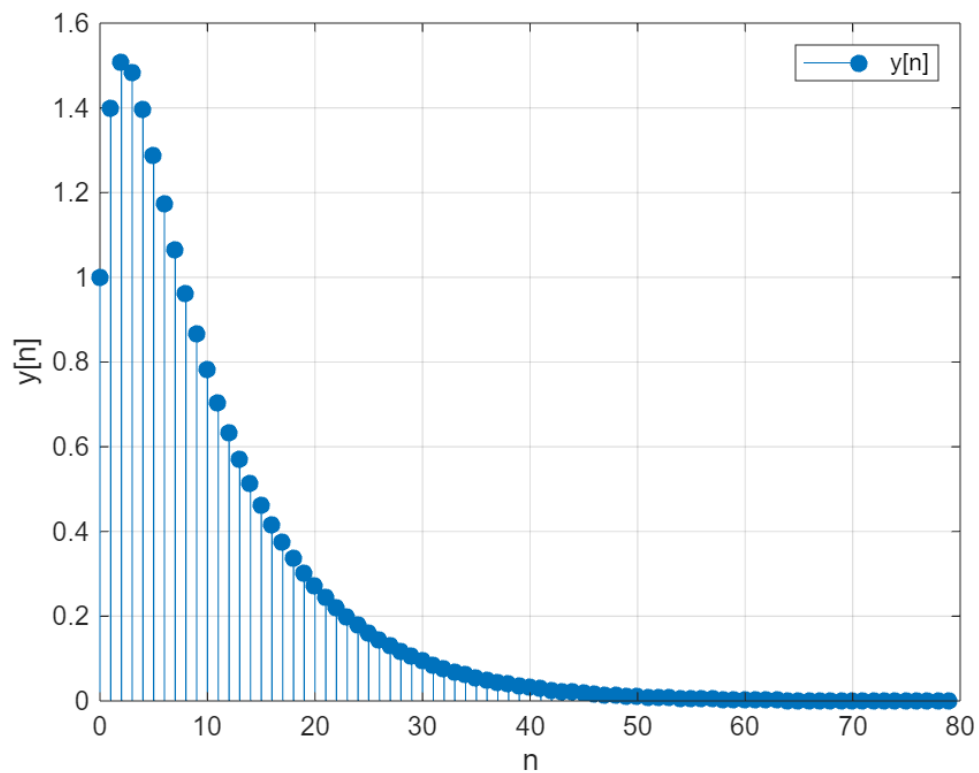
```
xn4 = 0:79;  
  
xn = 0.9.^xn4;  
  
hn = 0.5.^xn4;  
  
zn = conv([xn xn],hn);  
  
time2 = 0;  
  
for i = 0:1:49  
  
    f = @(x)(conv([x xn],hn));  
  
    t_timeie2 = timeit(f);  
  
    time2 = time2+t_timeie2;  
  
end  
  
yn=zn(81:160);  
  
stem(xn4,yn,'filled');  
  
grid on  
  
xlabel('n');
```

```
ylabel('y[n]')

legend('y[n]')

fprintf('执行时间（秒）： %.6f\n', time2/50);

f80c = time2/50
```



执行时间（秒）： 0.000002

f80c = 2.0495e-06

lab result:

figure is drawn.

3.10 g

(g). Assume $x[n]$ and $h[n]$ are defined as in Part (e). Use the periodic convolution property of the DTFS to implement the periodic convolution². Namely, compute the DTFS coefficients of both $x[n]$ and $h[n]$ using `fft` as described in Tutorial 3.1. Then, use the periodic convolution property to form the DTFS coefficients of $y[n]$. Finally, synthesize $y[n]$ from the DTFS coefficients using `ifft`. The `ifft` algorithm is nearly identical to the FFT, and also requires $O(N \log N)$ operations for a signal with fundamental period N . To check the validity of your implementation, plot $y[n]$ for $0 \leq n \leq N_y - 1$ and compare this signal to that computed in Part (e). Remember, as discussed in Tutorial 3.1, the signal $y[n]$ might have a small imaginary component due to numerical round-off errors. Store the total number of operations required to compute $y[n]$ in `f40f`, and remember to set `flops(0)` after creating the representations of $x[n]$ and $h[n]$ in MATLAB.

It is required to calculate $h[n]$ and $x[n]$ using FFT to calculate the DTFS of both, and then calculate the DTFS of $y[n]$ through the convolution coefficient of DTFS, and then restore $y[n]$ through the inverse transformation `ifft`, draw a graph and compare with e, and store the calculation time.

Analysis:

Just follow the instructions of the question.

```
N = 40;  
  
n = 0:N-1;  
  
x = (0.9).^n;  
  
h = (0.5).^n;  
  
tic;  
  
X = fft(x);  
  
H = fft(h);  
  
Y = X .* H;  
  
y = ifft(Y);  
  
time_elapsed1 = toc;  
  
num_ops = 2 * N * log2(N);  
  
fprintf('总操作数: %d\n', num_ops);
```

```
fprintf('执行时间 (秒): %.6f\n', time_elapsed1);
```

```
figure;
```

```
subplot(3,1,1);
```

```
stem(n, x, 'filled');
```

```
title('周期信号 x[n]');
```

```
xlabel('n');
```

```
ylabel('x[n]');
```

```
subplot(3,1,2);
```

```
stem(n, h, 'filled');
```

```
title('周期信号 h[n]');
```

```
xlabel('n');
```

```
ylabel('h[n]');
```

```
subplot(3,1,3);
```

```
stem(n, real(y), 'filled');
```

```
title('周期卷积 y[n]');
```

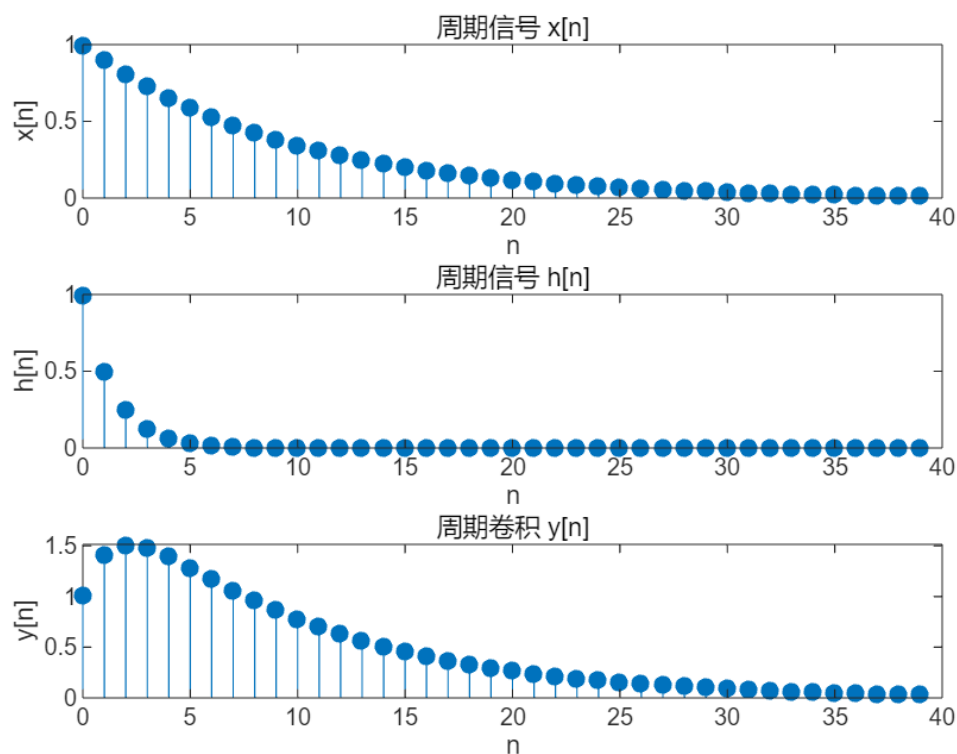
```
xlabel('n');
```

```
ylabel('y[n]');
```

```
f40f = time_elapsed1
```

总操作数: 4.257542e+02

执行时间 (秒): 0.004835



f40f = 0.0048

lab result:

As shown in the figure, the image and time are shown above, and the image is almost the same as the e question

3.10 h

(h). Repeat Part (g) for $N = 80$, again plotting a period of $y[n]$ and storing the number of operations in **f80f**. Again check the validity of your implementation by comparing $y[n]$ with that computed in Part (f).

The problem requires the calculation of $h[n]$ and $x[n]$. When $N=80$, the DTFS of both are calculated using FFT, and then the DTFS of $y[n]$ is calculated through the convolution coefficient of DTFS, and $y[n]$ is restored through inverse transformation, and the calculation time is stored.

Analysis:

Just follow the instructions of the question

```
N = 80;
```

```
n = 0:N-1;
```

```
x = (0.9).^n;

h = (0.5).^n;

f_lops = 0;

tic;

X = fft(x);

H = fft(h);

Y = X .* H;

y = ifft(Y);

time_elapsed2 = toc;

num_ops = 2 * N * log2(N);

fprintf('总操作数: %d\n', num_ops);

fprintf('执行时间 (秒): %.6f\n', time_elapsed2);

f_80f = num_ops;

figure;

subplot(3,1,1);

stem(n, x, 'filled');

title('周期信号 x[n]');

xlabel('n');

ylabel('x[n]');

subplot(3,1,2);

stem(n, h, 'filled');

title('周期信号 h[n]');

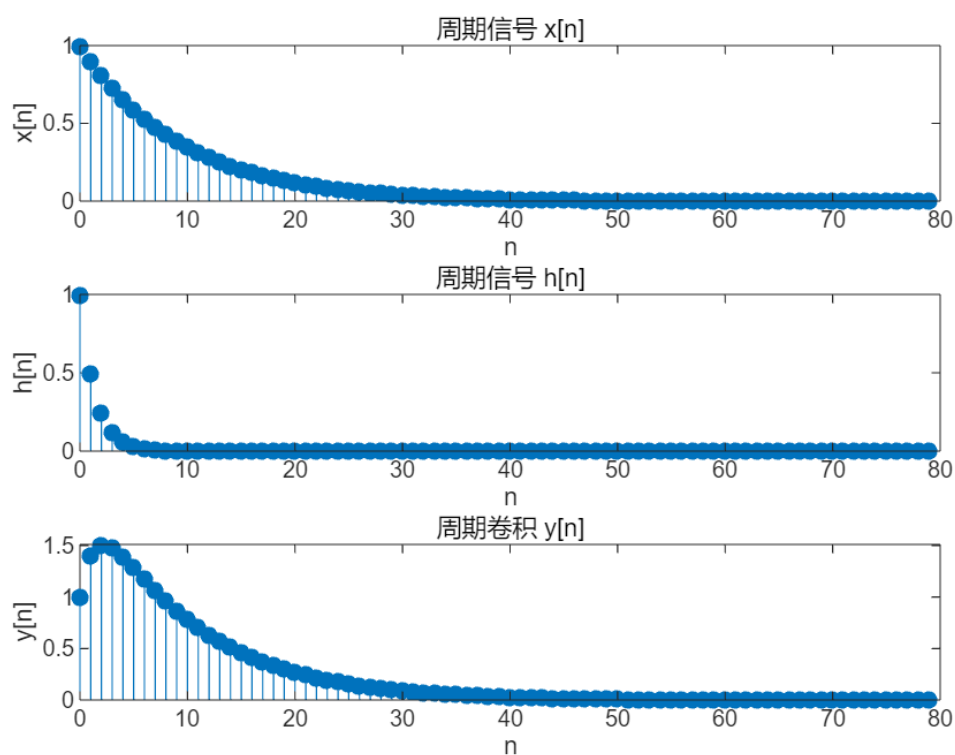
xlabel('n');

ylabel('h[n]');
```

```
subplot(3,1,3);  
  
stem(n, real(y), 'filled');  
  
title('周期卷积 y[n]');  
  
xlabel('n');  
  
ylabel('y[n]');  
  
f80f = time_elapsed2
```

总操作数: **1.011508e+03**

执行时间 (秒): **0.004218**



f80f = 0.0042

lab result:

figure is drawn.

3.10 i

(i). Compute the ratios of f_{40c} to f_{40f} and f_{80c} to f_{80f} . How do these ratios compare for $N = 40$ and $N = 80$? Which method of convolution is more efficient for each value of N ? Which method would you choose for $N > 80$? Justify your answer.

They want to compute f_{40c}/f_{40f} and f_{80c}/f_{80f} , and figure out which algorithm is more efficient for $N=40$ or $N=80$. And predict which algorithm is better when $N>80$.

Analysis:

Just compare.

```
fprintf('For N = %d:\n', 40);

fprintf(' Direct convolution operations: %f\n', f40c);

fprintf(' FFT convolution operations: %f\n', f40f);

fprintf(' Ratio (f_conv / f_fft): %f\n', f40c / f40f);

if(f40c/f40f <1)

    fprintf('conv is better');

else

    fprintf('fft is better');

end

fprintf('\n');

fprintf('For N = %d:\n', 80);

fprintf(' Direct convolution operations: %f\n', f80c);

fprintf(' FFT convolution operations: %f\n', f80f);

fprintf(' Ratio (f_conv / f_fft): %f\n', f80c / f80f);

if(f40c/f40f <1)
```



```
fprintf('conv is better');  
  
else  
  
    fprintf('fft is better');  
  
end  
  
fprintf('\n');
```

For N = 40:

Direct convolution operations: 0.000001

FFT convolution operations: 0.011878

Ratio (f_conv / f_fft): 0.000107

conv is better

For N = 80:

Direct convolution operations: 0.000002

FFT convolution operations: 0.004218

Ratio (f_conv / f_fft): 0.000470

conv is better

lab result:

Both values are much less than 1, so it is better to convolve the original signal directly. So when $N > 80$, it is better to use convolution.

Note: Please indicate meaning of the symbols in all expressions. Please indicate the coordinate and unit in all figures.

Experience

1. Use graphical methods to deepen understanding. MATLAB offers powerful plotting capabilities that help visualize spectral analysis and understand the transition between time and frequency domains. We use Utilize visualization tools to grasp theoretical concepts better, allowing us to see the practical effects of mathematical formulas.
2. Use built-in MATLAB functions such as **freqz** to analyze the frequency response of LTI systems. Experiment by altering parameters and observe their impact on results to understand system characteristics. We use continuously experiment with different settings, leveraging MATLAB's computational power to find patterns.
3. Learn the basic theory of filters, refer to filter design cases in the MATLAB community, and practice. MATLAB's **fdatool** is a very useful auxiliary tool for filter design. When we designing filters, first clarify the design objectives and constraints, then gradually implement the design by learning the steps involved.

Score

100/100

字体：英文 Times new Roman;